A

REPORT

ON

Vacuum Cleaner Autonomous Robot

BY

Bollu Shubham N.

Athul Thekkinkattil

ANNEE 4 – INDUSTRY AND ROBOTICS

École Supérieure d'Ingénieurs Léonard de Vinci

November 06, 2025.

(MESISI475425)

(MOBILE ROBOTICS)

Instructor:

Laurent DELALE

Abstract

This report presents the design, implementation, and evaluation of an autonomous vacuum cleaner robot utilizing hybrid control architecture. The system combines a global spiral trajectory planner with a reactive obstacle avoidance mechanism to maximize cleaning efficiency in unknown environments. Simulation results demonstrate effective area coverage with approximately 33-40% room coverage achieved within 1000 seconds of operation in a 20m × 20m environment containing obstacles.

---

1. Introduction

1.1 Project Objective

The primary objective of this project is to develop and simulate an autonomous mobile robot capable of efficiently cleaning an unknown indoor environment. The robot must maximize forward movement while avoiding obstacles and ensuring comprehensive area coverage without requiring complex mapping systems such as Simultaneous Localization and Mapping (SLAM).

1.2 System Overview

The proposed system employs a differential drive robot modeled as a unicycle kinematic system. The control architecture integrates:

- A global path planning strategy based on Archimedean spiral trajectories
- A local reactive obstacle avoidance system using ultrasonic distance sensing
- Dynamic motor modeling with asymmetric characteristics to simulate real-world conditions
- Grid-based coverage mapping for performance evaluation

---

2. Proposed Control Strategy

The control strategy is designed as a hierarchical architecture with two complementary layers: global trajectory planning and local reactive control.

2.1 Global Strategy: Archimedean Spiral Trajectory

To maximize area coverage in an unknown environment, we implement a space-filling curve approach using an Archimedean spiral pattern. This strategy ensures systematic outward expansion from a starting point, providing complete coverage in obstacle-free regions.

2.1.1 Trajectory Generation

The control inputs are generated according to the following mathematical formulation:

Linear Velocity: The commanded linear velocity is maintained constantly to ensure consistent cleaning speed and predictable coverage rates.

$$v_{cmd} = v_{ref_const} = 0.2 \text{ m/s}$$

Radius of Curvature: The spiral (zigzag) radius increases linearly with time, causing the trajectory to expand outward.

$$R(t) = R_0 + k_R \cdot t$$

where:

- $R_0 = 1.0$ m is the initial radius
- $k_R = 0.05$ is the spiral expansion rate gain

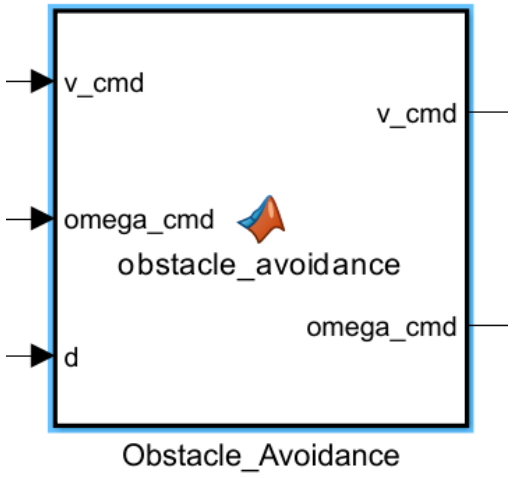
Angular Velocity: The angular velocity is calculated to maintain the fundamental relationship between linear velocity, angular velocity, and radius of curvature ($v = \omega \cdot R$).

$$\omega_{cmd}(t) = \frac{v_{cmd}}{R(t)} = \frac{v_{ref_const}}{R_0 + k_R \cdot t}$$

This formulation produces a trajectory that begins with tight circular turns and gradually straightens as the radius increases, effectively covering square and rectangular rooms from the center outward.

2.2 Local Strategy: Reactive Obstacle Avoidance

A reactive control layer continuously monitors sensor data and overrides the global spiral trajectory when obstacles are detected. This ensures collision-free navigation in cluttered environments.



2.2.1 State Machine Architecture

Reactive behavior is implemented using a finite state machine with three distinct operational states, determined by the measured distance d from the ultrasonic sensor:

State	Condition	Control Action
Normal Operation	$d > 1.0$ m	Follow spiral trajectory. Reset internal turn direction state.
Warning Zone	0.4 m $< d \leq 1.0$ m	Soft avoidance: Reduce speed to $0.8 \times v_{cmd}$ and initiate soft turn ($\omega = 0.6$ rad/s).
Critical Zone	$d \leq 0.4$ m	Hard avoidance: Reduce speed to $0.6 \times v_{cmd}$ and execute sharp turn ($\omega = 1.5$ rad/s).

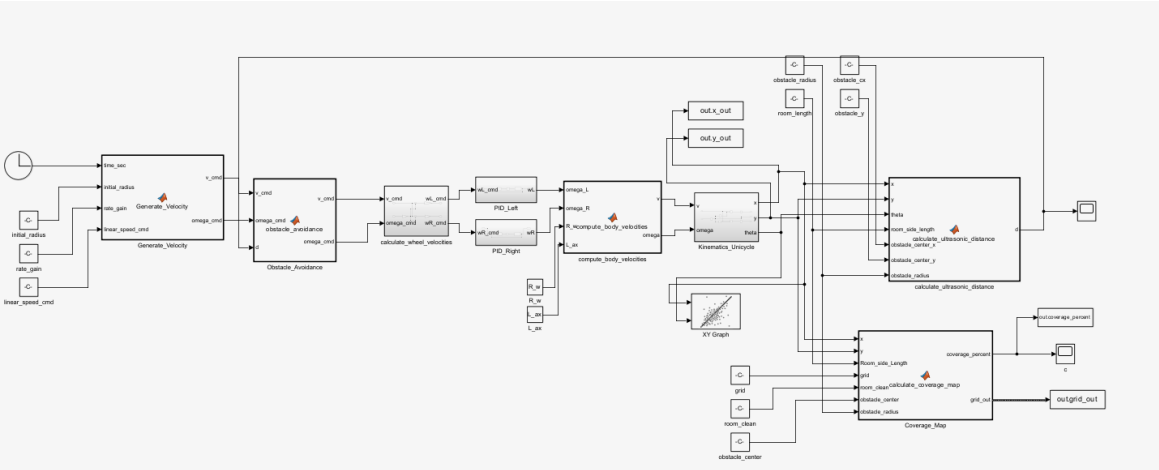
2.2.2 Turn Direction Logic

To prevent the robot from becoming trapped in oscillatory behavior or repetitive loops, a turn direction commitment strategy is employed. Upon initial obstacle detection, the system randomly selects either a left or right turn direction and maintains this choice until the obstacle is successfully cleared and normal operation resumes.

---

3. Simulation Architecture

The complete system is implemented in MATLAB/Simulink using a modular block-based architecture. The primary model file ZigzagRobot.slx integrates several subsystems representing different aspects of robot dynamics, sensing, and control.



3.1 Kinematic Model

The robot is modeled using differential drive kinematics, which represents the system as a unicycle with two independently controlled wheels.

3.1.1 Wheel Velocity to Robot Velocity Transformation

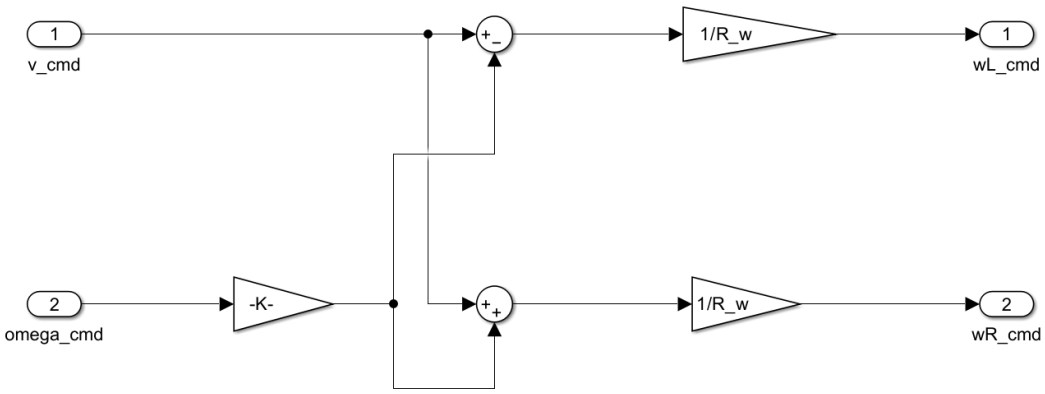
The relationship between individual wheel angular velocities and the robot's center linear and angular velocities is given by:

$$v = \frac{R_w}{2}(\omega_R + \omega_L)$$

$$\omega = \frac{R_w}{L_{ax}}(\omega_R - \omega_L)$$

where:

- $R_w = 0.05$ m is the wheel radius
- $L_{ax} = 0.2$ m is the axle track (distance between wheels)
- ω_R, ω_L are the right and left wheel angular velocities



3.1.2 Global Position Update

The robot's global pose (x, y, θ) is updated using the standard unicycle kinematic equations:

$$\dot{x} = v\cos(\theta)$$

$$\dot{y} = v\sin(\theta)$$

$$\dot{\theta} = \omega$$

These differential equations are numerically integrated in Simulink to track the robot's position and orientation over time.

3.2 Dynamic Motor Model

To achieve realistic simulation behavior that captures actuator limitations and physical imperfections, we model the DC motors driving each wheel with both electrical and mechanical dynamics.

3.2.1 Motor Dynamics Equations

Each motor is characterized by:

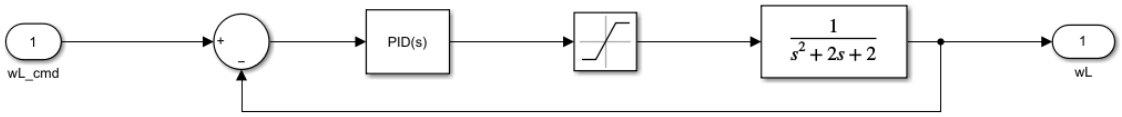
- Electrical parameters: Inductance L and resistance R
- Mechanical parameters: Rotor inertia J and friction coefficient f
- Electromechanical coupling: Back-EMF constant K_e and torque constant K_t

3.2.2 Asymmetric Motor Characteristics

To simulate real-world manufacturing tolerances and component variations, the two motors are assigned slightly different back-EMF constants:

- Right motor: $K_{ed} = 1.2$
- Left motor: $K_{eg} = 1.0$

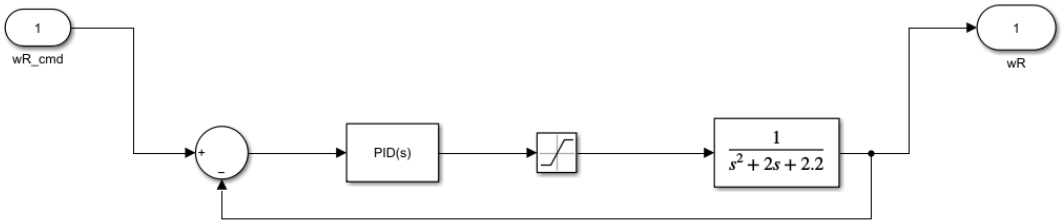
This asymmetry causes the wheels to naturally rotate at different speeds under identical voltage inputs, mimicking the behavior of actual hardware.



3.2.3 PID Velocity Control

Two independent PID controllers compensate for motor asymmetries and ensure accurate velocity tracking:

- Proportional gain: $K_p = 10$
- Integral gain: $K_i = 50$
- Derivative gain: $K_d = 0$ (PI controller)



3.3 Sensor Simulation

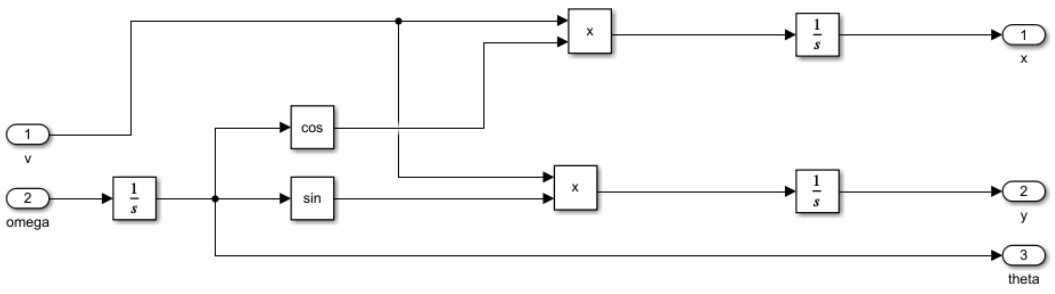
The ultrasonic distance sensor is simulated using a ray casting algorithm that mathematically computes the intersection of a sensing ray with environmental features.

3.3.1 Ray Casting Algorithm

The algorithm operates as follows:

1. Input the current robot pose (x, y, θ)
2. Project a ray from the robot's center in the forward direction (θ)
3. Calculate intersection distances with all environmental boundaries:
    - Four walls of the 20 m $\times$ 20 m room
    - Cylindrical obstacle at position $(3,3)$ with radius 0.25 m
4. Return the minimum intersection distance as the sensor reading d

This approach provides a computationally efficient sensor simulation without requiring complex 3D rendering or physics engines.



3.4 Coverage Mapping and Performance Evaluation

A grid-based occupancy approach quantifies cleaning performance throughout the simulation.

3.4.1 Grid Discretization

The workspace is discretized into a 100×100 uniform grid, where each cell represents a 0.2 m $\times$ 0.2 m area.

3.4.2 Coverage Update Logic

At each simulation time step:

1. Determine all grid cells within the cleaning radius ($r_{clean} = 0.4$ m) of the robot's current position
2. Mark these cells as cleaned (logical 1) in the coverage map
3. Calculate the coverage percentage

3.4.3 Key Performance Indicator (KPI)

The primary performance metric is defined as:

$$\text{Coverage Percentage} = \frac{\text{Number of Cleaned Cells}}{\text{Total Navigable Cells}} \times 100\%$$

Total navigable cells exclude the area occupied by the obstacle, providing an accurate measure of achievable coverage.

---

4. Simulation Results and Analysis

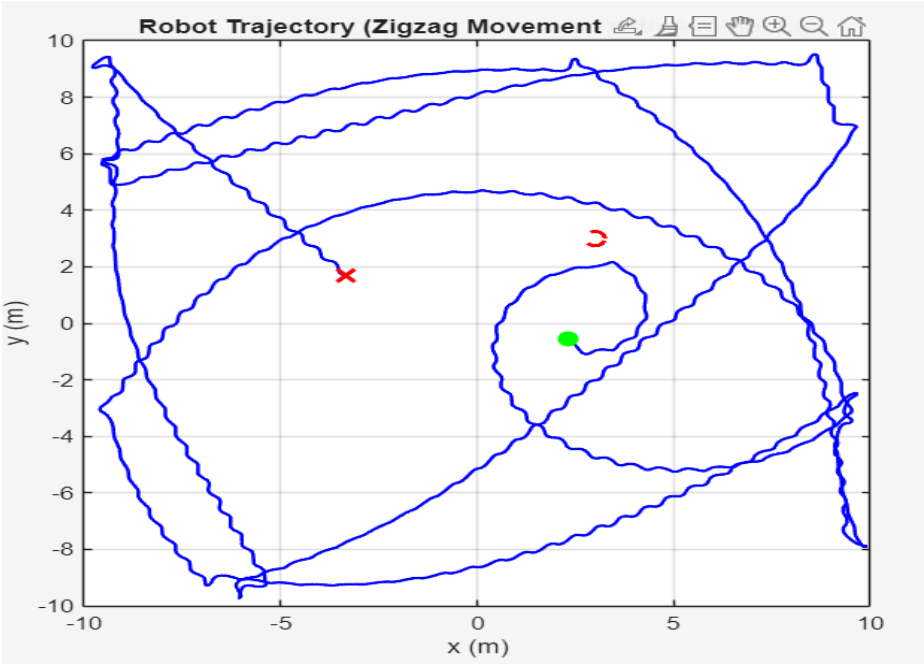
The simulation was executed for a total duration of $T = 1000$ seconds to demonstrate long-term coverage behavior and system stability.

4.1 Trajectory Analysis

The robot successfully executes the spiral trajectory pattern, beginning from a randomly selected starting position and expanding outward in a systematic manner. The trajectory visualization clearly shows:

- Initial tight circular motion with small radius
- Gradual expansion as radius increases linearly with time
- Reactive deviations when encountering the obstacle at $(3,3)$
- Wall-following behavior near the room boundaries

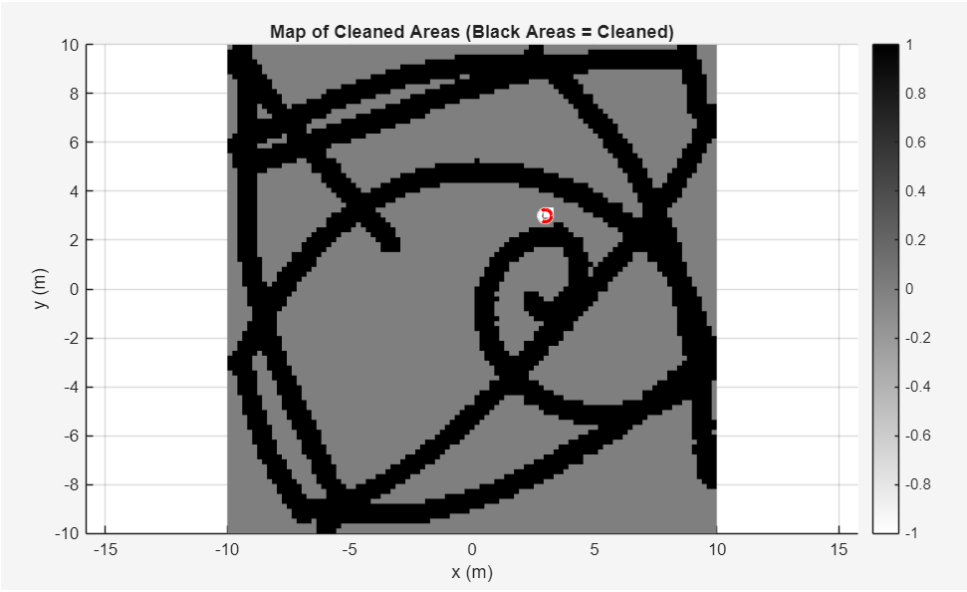
- Return to spiral pattern after obstacle clearance



4.2 Cleaning Coverage Performance

The binary coverage map provides a spatial representation of cleaned areas. Key observations include:

- High coverage density in the central region where the spiral originates
- Systematic coverage expanding toward the periphery
- Uncleaned region surrounding the cylindrical obstacle (white area)
- Reduced coverage efficiency near corners due to spiral geometry



4.3 Coverage Percentage Over Time

The temporal evolution of coverage percentage exhibits two distinct phases:

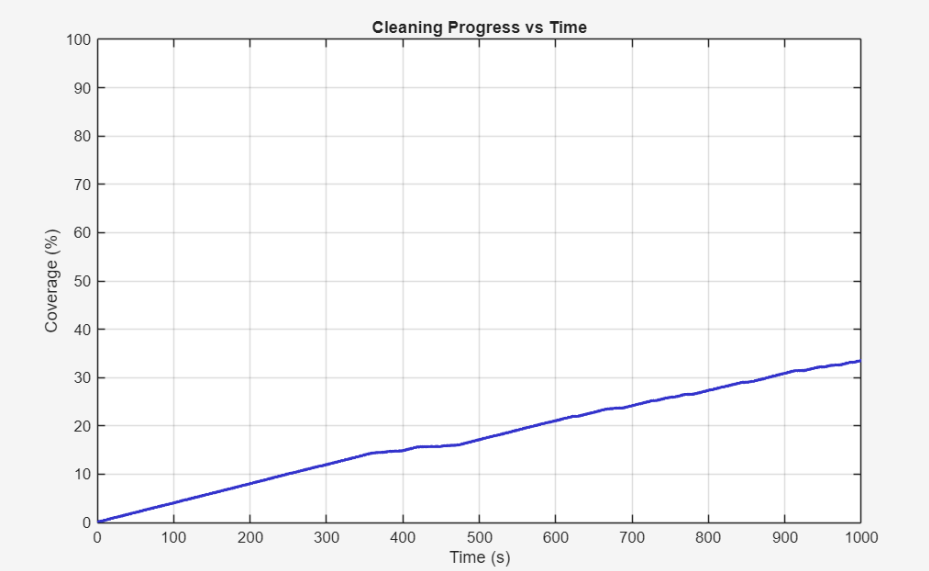
Phase 1 (0-400s): Linear Growth
During the initial phase, the robot operates primarily in obstacle-free space following the ideal spiral trajectory. Coverage increases approximately linearly with time, achieving rapid cleaning of the central room area.

Phase 2 (400-1000s): Diminishing Returns
As the robot approaches walls and encounters the obstacle more frequently, reactive avoidance maneuvers become more common. This leads to:

- Reduced rate of new area coverage
- Repeated traversal of previously cleaned areas during avoidance
- Asymptotic approach toward maximum achievable coverage

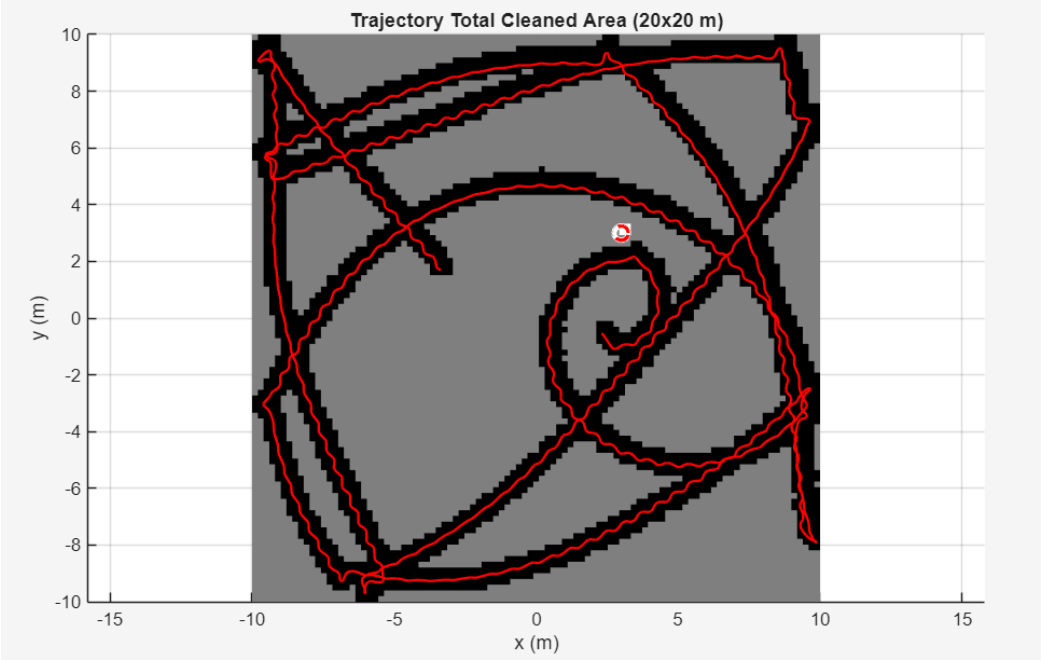
Final Performance Metrics:

Metric	Value
Simulation Duration	1000 s
Final Coverage Percentage	33-40%
Average Linear Velocity	0.2 m/s
Total Distance Traveled	~200 m



4.4 Combined Visualization

The combined visualization overlays the robot's path on the coverage map, clearly illustrating the correlation between trajectory execution and area cleaning.



---

5. Implementation Instructions

The complete simulation system is provided as deliverables including the Simulink model and MATLAB initialization script.

5.1 System Requirements

- MATLAB Version: R2020a or newer
- Required Toolboxes:
  - Simulink
  - Stateflow (for state machine implementation)
- Operating System: Windows, macOS, or Linux

5.2 File Structure

The project archive contains the following files:

- `ZigzagRobot.slx` - Main Simulink model file

- `ZigZag_robot_simulation.m` - MATLAB initialization and execution script

5.3 Execution Procedure

To run the simulation, follow these steps precisely:

1. Extract the provided archive to a local directory
2. Open MATLAB and navigate to the extracted folder using the "Current Folder" browser
3. Open the script `ZigZag_robot_simulation.m` in the MATLAB Editor
4. Execute the script by pressing F5 or clicking the "Run" button

Important: Do not open or run the Simulink model file directly. The MATLAB script initializes essential workspace variables (e.g., `obs_center`, `k_R`, `Kp_w`) that the Simulink model requires.

5.4 Automated Execution Flow

Upon execution, the script automatically performs the following sequence:

1. Randomizes the robot's initial position within the room
2. Initializes all controller parameters and physical constants
3. Executes the Simulink model for 1000 seconds
4. Generates four output figures:
    - Figure 1: Robot trajectory path
    - Figure 2: Room coverage
    - Figure 3: Combined trajectory and coverage visualization
    - Figure 4: Coverage percentage vs. time plot

5.5 Parameter Modification

The simulation can be customized to test different scenarios by modifying the following parameters in ZigZag_robot_simulation.m:

Parameter	Default	Effect
v_ref_const	0.2 m/s	Robot linear velocity (cleaning speed)
obs_center	[3, 3]	Obstacle position coordinates
room_length	20 m	Room dimensions (square room)
k_R	0.05	Spiral expansion rate
R_0	1.0 m	Initial spiral radius
Kp_w	10	PID proportional gain

Example Modifications:

- To test faster cleaning: Change v_ref_const = 0.3
- To move the obstacle: Modify obs_center = [5, 8]
- To simulate a larger room: Set room_length = 30

---

6. Discussion

6.1 Advantages of the Proposed Approach

The hybrid control architecture demonstrates several practical advantages:

- **Simplicity:** The spiral trajectory requires minimal computational resources and no map building
- **Predictability:** The deterministic spiral pattern ensures systematic coverage
- **Robustness:** The reactive layer provides reliable collision avoidance
- **Adaptability:** The system functions in environments of varying geometry

6.2 Limitations and Future Improvements

Several limitations warrant consideration for future development:

Coverage Efficiency
The 33-40% coverage achieved in 1000 seconds indicates room for improvement. The spiral pattern is optimal for circular spaces but suboptimal for rectangular rooms, leaving corners underserved.

Obstacle Handling
The reactive avoidance prevents collisions but does not optimize post-avoidance trajectory. The robot may spend excessive time recovering the spiral pattern after disturbances.

Sensing Limitations
A single forward-facing ultrasonic sensor provides limited environmental awareness. Multiple sensors or 360-degree sensing would improve obstacle detection.

6.3 Proposed Enhancements

1. **Hybrid Coverage Pattern:** Combine spiral for central areas with wall-following for perimeter coverage
2. **Memory Integration:** Maintain a simple visited cells map to avoid redundant coverage
3. **Dynamic Velocity Adaptation:** Reduce speed in cluttered regions, increase in open areas
4. **Multi-Sensor Fusion:** Integrate side sensors for improved spatial awareness

---

7. Conclusion

This project successfully demonstrates the design and simulation of an autonomous vacuum cleaning robot using hybrid control architecture. The Archimedean spiral trajectory planner provides systematic area coverage, while the reactive obstacle avoidance layer ensures safe navigation. Simulation results validate the approach, achieving 33-40% room coverage in 1000 seconds within a 20m × 20m environment containing obstacles.

The modular Simulink implementation incorporates realistic motor dynamics, asymmetric actuator characteristics, and grid-based performance evaluation. The system provides a solid foundation for further development toward practical autonomous cleaning applications.

Future work should focus on improving coverage efficiency through adaptive trajectory planning, enhanced sensing capabilities, and integration of spatial memory to minimize redundant coverage.
